

Can Retrieval-Augmented Few-Shot Prompting Match LoRA Fine-Tuning for Issue Report Classification? An Empirical Study

Anonymous Author(s)
Anonymous Affiliation(s)

Abstract—Issue report classification (IRC) accelerates modern software maintenance, yet it remains a challenging task. State-of-the-art automated IRC methods fine-tune Large Language Models (LLMs) with Low-Rank Adaptation (LoRA), which is computationally expensive and must be repeated to incorporate newly labeled issue reports or whenever the model is swapped. We empirically investigate the hypothesis that low-cost, retrieval-augmented generation (RAG)-based approaches can achieve IRC performance competitive with the LoRA fine-tuning baseline. If validated, this would introduce the possibility of cost-efficient IRC solutions, an avenue that, to our knowledge, remains unexplored in the literature. To test this hypothesis, we conduct a case study on the Qwen2.5-Instruct family, a set of open-source models with strong public-benchmark performance, evaluated at four parameter sizes to analyze the effect of model scale. We propose three retrieval-based methods, VOTAG, RAGTAG, and BRAGTAG, each of which works by retrieving the top- k most similar issues from an index of labeled issues and reasoning on them. Specifically, VOTAG assigns the label via a similarity-weighted vote over the retrieved neighbors; RAGTAG presents the neighbors as classified examples in a few-shot LLM prompt; and BRAGTAG extends RAGTAG with a more balanced retrieval scheme that better covers the label space. We evaluate all three methods against a LoRA fine-tuning baseline on an 11-project benchmark, under both *project-specific* and *project-agnostic* data scope settings. With project-specific data, RAGTAG outperforms fine-tuning by 0.021–0.040 macro F_1 across all model sizes. Fine-tuning on cross-project data beats project-specific RAGTAG by only 0.029 aggregate macro F_1 despite $11\times$ more labeled data. The balanced retrieval approach, BRAGTAG, closes this performance gap, achieving statistical equivalence with fine-tuning when using a retrieval fallback for invalid LLM outputs. **Per-class precision and recall show that these gains come from correcting the models’ tendency to over-predict bug, a bias that LoRA fine-tuning retains.** Averaged across model sizes, retrieval-augmented few-shot prompting requires 33% less peak GPU memory than fine-tuning. Under our experimental setup, RAG-based approaches demonstrate competitive performance against LoRA fine-tuning, suggesting they represent a viable and cost-efficient alternative for IRC, one that the literature has yet to fully explore.

Index Terms—Issue classification, Retrieval-augmented generation, Fine-tuning, Large language models, Mining software repositories

I. INTRODUCTION

Issue Tracking Systems provide a structured way to communicate concerns related to software projects by allowing users to submit issue reports [1]. Issue reports can be classified with labels to help with triage. Effective issue report classification (IRC) results in faster issue resolution [2], and is crucial for

many software maintenance tasks, such as issue prioritization [3], issue assignment [4], and developer onboarding [5], [6]. However, studies show that labels are rarely applied in issue creation [7]–[9]. As a result, project maintainers must manually classify the issues for triage, which is laborious and time-consuming [10].

To address this problem, numerous automated methods have been proposed around supervised learning over three established issue report categories: *bug*, *feature*, and *question* [6], [7], [9], [11], [12]. Early work relied on classic machine learning (ML) techniques [7]–[10], while more recent studies fine-tune transformer models on labeled issue reports [11], [13]–[18]. However, these approaches require a substantial amount of training data to be effective [11], [13], [16], [19], [20]. The current state-of-the-art methods fine-tune Large Language Models (LLMs) using Low-Rank Adaptation (LoRA [21]) [6], [12], [19]. However, fine-tuning LLMs is computationally expensive and must be repeated whenever the model is swapped or new labeled issues are incorporated in the project [9], [10].

Prior work shows that placing a few demonstration examples in the prompt can boost an LLM’s performance on specialized tasks [22]–[26] through in-context learning [26]. Few-shot performance, however, depends on the choice of in-context examples. Studies show that using the query’s retrieved nearest neighbors as in-context examples yields strong few-shot performance [27], [28]. Additionally, the nearest neighbors tend to share the same label as the query [29], making them suitable in-context examples. The combination of these insights leads to retrieval-augmented few-shot prompting, where the classified issue reports most similar to a query are retrieved and used as its in-context examples. Since this method enhances performance without model training or weight updates, it has the potential to serve as a cost-efficient alternative to LoRA fine-tuning for IRC. Yet both a detailed empirical evaluation of this approach for IRC and its comparison against fine-tuning remain underexplored. To address these gaps, we conduct an empirical study that examines whether it is possible for retrieval-augmented few-shot prompting to achieve IRC performance competitive with LoRA fine-tuning in practice.

An important factor in retrieval-augmented few-shot performance is the number of provided examples: too few may not provide enough context, and too many can add noise [27]. Since the in-context examples are the query’s nearest neighbors, they can classify the query under the standard k -nearest-

neighbor (KNN) framework [29], [30]. Therefore, the point where adding neighbors stops improving the retrieval-only classification informs an upper bound for the number of in-context examples. Additionally, retrieval-only IRC provides a non-parametric baseline that isolates the LLM’s contribution from that of retrieval in retrieval-augmented few-shot prompting. To our knowledge, no prior study has explored such a baseline for IRC. Therefore, we pose our first research question:

RQ1. How effectively can retrieval alone classify issue reports, and at what point does adding more retrieved neighbors stop helping?: We answer this question by applying a standard distance-weighted KNN classification [29], [30]. Given a query issue, its top- k most similar issue reports (nearest neighbors) are first retrieved from an embedded index of existing issue reports. Next, the label is predicted with a similarity-weighted vote over the neighbors. We evaluate this method over a wide range of top- k values to measure its peak performance and identify the point where additional neighbors are not beneficial (best k value). With the best k value established, we ask:

RQ2. How well does retrieval-augmented few-shot prompting classify issue reports, and how does performance vary with the number of in-context examples?: We answer this question by presenting the same top- k nearest issue reports as classified examples in a few-shot prompt, instructing the model to predict a label for the query issue. $k=0$ results in a zero-shot LLM baseline similar to prior work [31]. We evaluate this approach with different model scales over a range of k values bounded by RQ1. RQ2 evaluations reveal that retrieval-augmented few-shot has the weakest performance on the question class, and question issues are frequently misclassified as bugs. Zero-shot prompting results indicate that the models are already biased towards question-to-bug misclassifications even before the examples are presented. We hypothesize that bug-labeled neighbors retrieved for true-question queries reinforce this misclassification. This motivates our next research question:

RQ3. Can a balanced retrieval intervention reduce question-to-bug misclassification and improve retrieval-augmented few-shot performance?: We answer this question by exploring a more balanced retrieval technique that removes bug-labeled neighbors from the prompt whenever the retrieved neighbors’ label distribution suggests the query is likely a question. We evaluate it over the same k values and model scales used for RQ2. For brevity, we refer to the retrieval-only classifier (RQ1) as **V**oting-based **T**AGging (VOTAG), the retrieval-augmented few-shot approach (RQ2) as **R**etrieval-Augmented **G**enerative **T**AGging (RAGTAG), and its balanced variant (RQ3) as **B**alanced RAGTAG (BRAGTAG). We compare these methods against the established LoRA [21] fine-tuning baseline [6], [12], [19] to answer our final research question:

RQ4. How do retrieval-based methods compare against LoRA fine-tuning for IRC in classification performance

and computational cost?: We conduct this comparison using the Qwen2.5-Instruct model family at 3B, 7B, 14B, and 32B parameters [32] to analyze the effect of model scale in our evaluations. We use Heo et al.’s [19] 11-project dataset for our experiments and adopt its two evaluation settings: a *project-specific* (PS) configuration where retrieval/training is over each project’s individual data split in isolation, and a *project-agnostic* (PA) configuration where retrieval/training is over data from the 11 projects pooled together.

Results show that retrieved few-shot examples consistently improve IRC performance. Even a single retrieved example outperforms the zero-shot baseline across all model sizes (Section V-B). LLM-based reasoning also consistently improves over pure retrieval: every RAGTAG configuration outperforms VOTAG’s best result. Notably, however, VOTAG comes within 0.018 macro F_1 of Qwen-3B zero-shot performance (Section V-A). Retrieval-augmented few-shot prompting is also competitive with LoRA fine-tuning. Using only each project’s own labeled issue reports under the PS data scope, RAGTAG outperforms project-specific LoRA fine-tuning by 0.021–0.040 macro F_1 across all model sizes (Section V-D). LoRA fine-tuning only catches up when trained under the PA data scope, which pools $11\times$ more labeled data across projects. Even then, RAGTAG remains competitive: aggregated across all model sizes, project-agnostic LoRA fine-tuning leads project-specific RAGTAG by only 0.029 macro F_1 , and the two methods are statistically tied at Qwen-3B and Qwen-32B. BRAGTAG further narrows this gap by reducing question-to-bug misclassifications without sacrificing performance on the other labels (Section V-C). [Precision and recall show that this correction targets a bias shared by all LLM-based methods: zero-shot and fine-tuned models label 40–56% of issues as bug against a true share of 33%, whereas BRAGTAG brings this share to 31–39% and raises question recall by 0.07–0.15.](#) The aggregate macro F_1 difference between BRAGTAG and LoRA fine-tuning is -0.010 (BRAGTAG – LoRA fine-tuning). Applying a VOTAG fallback to invalid LLM outputs across all methods further tightens the comparison to statistical equivalence, passing *two one-sided tests (TOST)* [33] at $\delta=0.01$. Retrieval-augmented few-shot offers deployment advantages as well. Averaged across the four model sizes, RAGTAG and BRAGTAG require 33% less peak GPU memory than fine-tuning (Section V-D). Additionally, incorporating newly labeled issues and swapping the underlying model mid-production do not require retraining cycles.

Overall, our findings suggest that retrieval-augmented few-shot prompting can achieve IRC performance competitive with LoRA fine-tuning. [These findings are consistent across macro \$F_1\$, macro precision and recall, and per-class precision and recall.](#) This makes it a practical alternative worth considering, particularly when hardware resources are constrained, labeled data is limited, or the deployment requires frequent model or data updates. The main contributions are the following:

- Establishing a retrieval-only, non-parametric baseline for IRC that also serves as a fallback mechanism for invalid LLM predictions (VOTAG).

- Conducting the first systematic evaluation of retrieval-augmented few-shot prompting for IRC across different LLM scales and numbers of in-context examples (RAG-TAG).
- Proposing a balanced retrieval technique for few-shot prompting that reduces question-to-bug misclassifications without degrading performance on the other labels (BRAG-TAG), **raising question recall by 0.07–0.15 and correcting the models’ tendency to over-predict bug.**
- Empirically comparing these proposed methods against the established LoRA fine-tuning baseline on both classification performance and computational cost, in two data scope settings (PS and PA). We find that retrieval-augmented few-shot prompting is competitive with fine-tuning while using $11\times$ less labeled data per project and 33% less peak GPU memory.

The remainder of this paper is organized as follows: Section II reviews prior work. Section III explains our proposed methods and the fine-tuning baseline. Section IV details the experimental setup. Section V explains our evaluations and reports the results. Section VI discusses failure modes, practical implications, and future work. Section VII addresses the potential threats to validity. Finally, Section VIII concludes the paper.

II. RELATED WORK

Automated Issue Report Classification (IRC): Early IRC studies employed data mining and ML classifiers [7], [9], [10]. However, these methods yielded limited performance due to their shallow semantic understanding of issue report text [19]. To overcome this limitation, more recent work has adopted transformer models for their stronger contextual understanding [11], [13]–[16]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [11] and Trautsch et al. [14] fine-tuned RoBERTa [34] and seBERT [35] models, respectively, to classify issues into bugs, features, or questions. Although transformer-based approaches achieve strong results, they rely on large amounts of training data, which limits their generalizability to settings where labeled data is scarce [11], [13], [16], [19]. These challenges have motivated the exploration of LLMs for IRC. Their extensive pretraining allows these models to be applied with little or no task-specific training, reducing the dependence on the large labeled datasets that supervised classifiers require [1], [6], [12], [31], [36]. However, state-of-the-art LLM-based methods do not leverage this property. Instead, they fine-tune the LLM with Low-Rank Adaptation (LoRA [21]) on labeled issue datasets to achieve their best reported performance [6], [12], [19]. Although fine-tuning is effective, it reintroduces the dependency on labeled training data, remains computationally expensive, and must be repeated whenever the underlying model is swapped or as newly labeled issues accumulate in the project [9], [10].

Retrieval-Augmented Few-shot Prompting: Studies show that few-shot prompting can improve LLM performance in

specialized tasks due to in-context learning [22]–[26]. Few-shot prompting has also been applied in IRC. Colavito et al. [1], [31] randomly selected one or two labeled examples from their dataset and presented them in a few-shot prompt. Few-shot performance, however, is sensitive to the quality of in-context examples. Research shows that using retrieval techniques to select the most semantically relevant in-context examples can outperform random selection [27], [28]. Since retrieval-augmented few-shot prompting can enhance performance without the overhead of training, it can be a promising alternative to the LoRA fine-tuning IRC baseline. To the best of our knowledge, however, this method has not been systematically and empirically evaluated for IRC.

III. APPROACH

In this section, we describe the intuition behind and the implementation details of our three proposed retrieval-based IRC methods (VOTAG, RAGTAG, and BRAGTAG) as well as steps for faithfully reproducing the established LoRA fine-tuning baseline [6], [19]. Fig. 1 illustrates all four approaches to support the explanations. The remainder of the section is organized as follows: Section III-A explains the overall IRC objective. Section III-B details the indexing and retrieval pipeline shared by our three proposed retrieval-based methods. Section III-C presents VOTAG, our non-parametric method of classifying the query directly from its nearest neighbors. Section III-D describes RAGTAG, our few-shot prompting technique that presents the nearest neighbors as classified examples in the prompt. Section III-E elaborates on BRAGTAG’s balanced retrieval intervention applied to RAGTAG. Finally, Section III-F details the faithful reproduction of the fine-tuning baseline.

A. Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue’s title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [6], [7], [9], [11], [12].

B. Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., *title + description*). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate the embeddings, we use the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [37]. We selected this model due to its lightweight architecture, fast performance, and effectiveness on issue report text [38]. The embeddings are then indexed using the Faiss vector database library [39], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, Faiss calculates the cosine similarity of indexed issue reports to the query, and returns the top-K nearest neighbors, ordered by descending similarity.

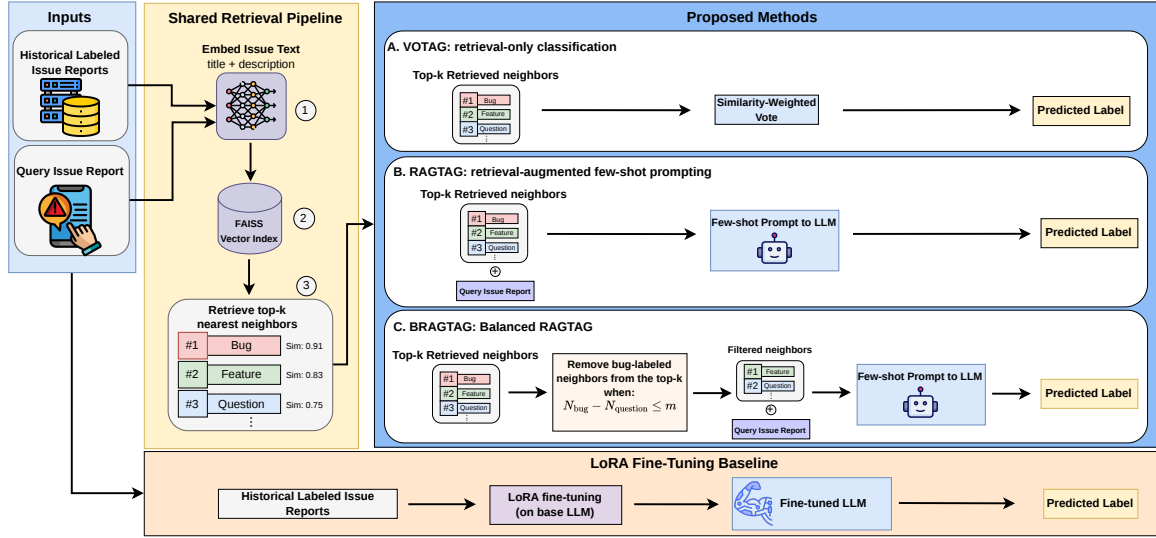


Fig. 1. Overview of the four IRC approaches evaluated in this paper: VOTAG, RAGTAG, BRAGTAG, and the LoRA fine-tuning baseline.

C. VOTAG

VOTAG is our proposed non-parametric retrieval IRC method. VOTAG uses the retrieval pipeline described in Section III-B to retrieve the top-K nearest neighbors of the query issue report, then predicts the final label with a similarity-weighted vote over their labels. This approach follows the standard K-nearest-neighbor classification setup [29], with neighbors weighted by their similarity to the query [30].¹ Formally, for a query issue q with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^K$, where n_i is the i -th nearest neighbor with label l_i and cosine similarity s_i to q , the predicted label $\hat{y}$ is given by:

$$\hat{y} = \arg \max_{c \in \mathcal{L}} \sum_{i=1}^K s_i \cdot \mathbf{1}[l_i = c], \quad (1)$$

where $\mathcal{L}$ is the label set, and $\mathbf{1}[l_i = c]$ is the indicator function that equals 1 when neighbor i has label c and 0 otherwise. For each label, VOTAG sums the cosine similarities of the neighbors that carry that label, and the $\arg \max$ operator returns the label with the largest total sum. VOTAG breaks ties by returning the label of the most similar neighbor in the tied classes.

D. RAGTAG

RAGTAG is our proposed retrieval-augmented few-shot prompting approach for IRC. RAGTAG uses the retrieval pipeline described in Section III-B to retrieve the top-K nearest neighbors to the query issue, and presents them as classified examples to the LLM in a few-shot prompt. This enables a more accurate classification, based on in-context learning [26]. The choice to present the top-K nearest neighbors as examples

¹Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

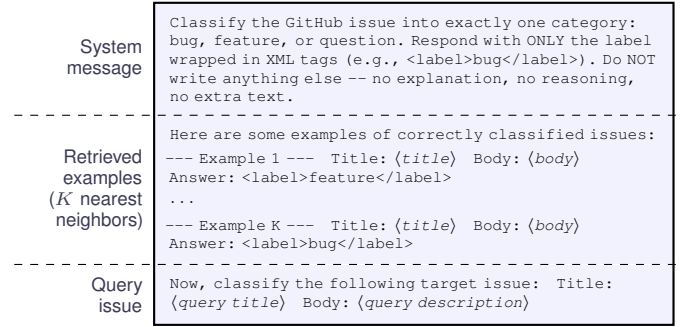


Fig. 2. The retrieval-augmented few-shot prompt structure used for RAGTAG and BRAGTAG.

is based on two insights from prior research: First, retrieval-based example selection outperforms random selection [27]. Second, the most similar neighbors tend to share the same labels [29]; therefore, they are informative for the classification. We create the few-shot prompt in three parts as shown in Fig. 2. First, the system message frames the classification task and the label space. Second, the top-K nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title + description + label*. Finally, the query issue is appended to the end of the prompt, ensuring it receives high attention from the LLM [40]. $K = 0$ produces a zero-shot prompting baseline, similar to prior work [31].

We evaluate RAGTAG at top-K values in $\{0, 1, 3, 6, 9, 12, 15\}$. Details for the K value grid selection are provided in Sections V-A and V-B. The maximum prompt sequence length is set to 8,192 tokens.² Roughly 30% of the context quota is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota,

²Maximum prompt sequence length is selected empirically. 8,192 tokens offered the best trade-off between classification accuracy and inference speed across our preliminary experiments.

they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving all titles and labels (if any). The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format. This technique enables easy detection of the predicted label from the LLM’s generation, and reduces variance in LLM outputs caused by prompt format sensitivity [41]. We use a regular expression to extract the predicted label. Unparseable outputs are marked as *invalid*. Finally, models are loaded with Unsloth³ for memory efficiency, and the Temperature is set at 0.1 for deterministic classifications.

E. Balanced RAGTAG (BRAGTAG)

Our empirical evaluations in Section V-B reveal that RAGTAG has the worst performance on the question label, and it frequently misclassifies true-question queries as bugs. Furthermore, LLM zero-shot results also show that the models are already biased towards predicting the bug label for true-question queries even before the examples are presented. Based on these insights, we hypothesize that presenting bug-labeled examples for true-question queries likely reinforces the question-to-bug misclassification trend. To address this, we propose a more balanced retrieval intervention, called balanced RAGTAG (BRAGTAG), that removes the bug-labeled neighbors whenever the query is a suspected question based on its nearest neighbors’ label distribution. Section V-B shows that true-question queries retrieve almost balanced bug and question neighbors. Due to this balanced distribution, BRAGTAG removes the bug-labeled examples from the prompt whenever $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the top- k and m is a small positive margin. The margin is empirically calibrated on a holdout validation set. The empirical motivations for BRAGTAG and its margin calibration are fully detailed in Section V-B. Other than the balanced retrieval intervention, BRAGTAG inherits RAGTAG’s full setup and configurations.

F. LoRA Fine-Tuning Baseline

Recent IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [6], [12], [19], using Low-Rank Adaptation (LoRA) [21]. In these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt that includes the issue’s title and description as input, and its label as output. We adopt the instruction-style prompt template used in these prior LoRA fine-tuning studies [6], [19] verbatim, rather than designing our own, so that the baseline faithfully reproduces their established methodology. At inference, the trained model receives the test issue in the same format, and predicts the empty label field. Regular expressions are used to extract the prediction from the output, and unparseable outputs are marked as *invalid*. We use this established methodology as our fine-tuning baseline.

Training hyperparameters follow prior work [6], [19]: The LoRA rank and alpha are set to 16, with a learning rate of 2×10^{-4} and the paged AdamW 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient accumulation steps of 16. The maximum prompt sequence length is set to 2,048 tokens, since 94.9% of the test issues in the dataset will not be truncated at this length. Issues that exceed this length get truncated from the right side of their description text to fit. Similar to RAGTAG and BRAGTAG, the models are loaded with Unsloth and the Temperature is set at 0.1.

IV. EXPERIMENTAL SETUP

Dataset and Settings.: We use the dataset introduced in Heo et al. [19] for our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible`, `bitcoin`, `dart-lang`, `roslyn`, `react`, `flutter`, `kubernetes`, `TypeScript`, `vscode`, `opencv`, and `tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug*, *feature*, and *question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project. We also adopt Heo et al.’s project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC performance. In the PS setting, each project’s train and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAGTAG and VOTAG only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline.

Models.: We use the Qwen2.5-Instruct model family [32] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model with different parameter sizes enables analysis of the effect of model scale in LLM-based IRC approaches.

Evaluation Metrics.: We report the macro-averaged F_1 over the three classes (*bug*, *feature*, *question*) as the headline metric and use it to select each method’s best configuration. We also report macro-averaged and per-class precision and recall. All metrics are calculated over the total 3,300-issue test set. We also report the rate of outputs that fail to parse to a valid label (*invalid rate*). Invalid outputs are counted as incorrect predictions. For statistical significance checks, we report paired bootstrap 95% confidence intervals on the macro F_1 differences between methods.

Hardware Setup.: The experiments were conducted on a machine with one NVIDIA L40 GPU (48 GB VRAM), 8 allocated CPU cores from an Intel Xeon Scalable processor, and 64 GiB of system RAM.

³<https://unsloth.ai>

V. EVALUATION

In this section, we present the results of our experiments and answer the four research questions posed in Section I, one per subsection.

A. RQ1: Effectiveness of Retrieval-Only IRC

We evaluate VOTAG’s performance on both PA and PS settings at k values from 1 to 30. The resulting macro F_1 scores are shown in Fig. 3 (left panel). VOTAG’s best macro F_1 is 0.595 ($k=15$) in PS and 0.604 ($k=16$) in PA. PA marginally outperforms PS at every k . This near similar performance is due to the large overlap of the top- k neighbors in both settings. Specifically, PA and PS share 84–90% of the top- k neighbors across $k \in [3, 30]$. Therefore, even when using the full 3,300-issue retrieval index, most of the retrieved neighbors are from the same project.

The question class consistently has the weakest performance among the three labels at every k in both settings. Additionally, questions are more frequently misclassified as bugs: 32% of question issues are predicted as bugs in both settings, while 18% (PS) and 17% (PA) are predicted as features. **More generally, VOTAG over-predicts bug, the only class whose recall (0.73) exceeds its precision (0.55–0.56) at every k (Fig. 3, right panel).**

Retrieval alone establishes a non-parametric baseline with a peak macro F_1 of 0.604 and bounds the useful number of in-context examples at $k \approx 15$. **Question is the weakest class: misclassified question issues are predominantly mislabeled as bugs, and retrieval over-predicts the bug class in general.**

B. RQ2: Effectiveness of Retrieval-Augmented Few-Shot IRC

We evaluate RAGTAG at k values $\{0, 1, 3, 6, 9, 12, 15\}$ on both PA and PS settings, where $k=0$ is the zero-shot baseline [31]. The selection of k values is informed by VOTAG’s peak performance around $k=15$ in the previous section. Fig. 4 (left panel) shows the macro F_1 of every model across this grid for both settings.

PS marginally outperforms PA at almost every (model, $k \geq 1$) pair evaluated despite using $11 \times$ less retrieval data per project (zero-shot is identical between settings since no retrieval is performed). The close performance gap is due to the same top- k neighbor overlap observed in the VOTAG analysis. Given its advantage, we conduct the rest of our RAGTAG analysis on the PS setting and report PS numbers throughout this section.

The k that maximizes macro F_1 grows with model size: $k=3$ for Qwen-3B, $k=6$ for Qwen-7B, and $k=12$ for both Qwen-14B and Qwen-32B. Larger models thus make better use of additional examples, although adding neighbors beyond $k=12$ brings no further gain and slightly degrades performance.

The LLMs benefit substantially from the labeled examples: every RAGTAG configuration with $k \geq 1$ outperforms the zero-shot baseline; at the best k , the gain averages +0.076 macro F_1 across the four models. Every configuration also outperforms VOTAG’s best result (0.604 at PA $k=16$), showing that adding LLM reasoning improves IRC on top of pure retrieval.

Similar to VOTAG, the question class consistently has the weakest performance among the three labels at every k and in both settings. **Per-class precision and recall (Fig. 4, right panels) show that this weakness is a recall deficit caused by question issues being misclassified as bug: question has the lowest recall of the three labels but never the lowest precision.** At zero-shot, across all four model sizes, 46–59% of true-question issues are predicted as bugs while only 5–16% are predicted as features. **These question-to-bug errors also account for roughly 80% of all issues incorrectly predicted as bug, so a single error type explains both the low recall of the question class and the low precision of the bug class.** Most of RAGTAG’s macro F_1 improvement over zero-shot comes from reducing these errors: averaged over the four models, moving from zero-shot to RAGTAG at the best k lowers the share of issues predicted as bug from 51% to 42%, raising bug precision from 0.58 to 0.66 and question recall from 0.35 to 0.51. RAGTAG reduces the rate of the question-to-bug errors but does not fully eliminate them: at each model’s best k , 26–36% of question issues are still predicted as bugs and only 9–15% as features.

These results show that the LLMs are already prone to misclassifying question issues as bugs, even before the top- k examples are used. Therefore, presenting bug-labeled examples for true-question query issues likely reinforces these misclassifications. Based on this intuition, we propose a more balanced retrieval technique as an intervention that removes the bug-labeled neighbors whenever the query is a suspected question. We base the suspicion signal on the nearest neighbors’ label distribution and calibrate it without including the test set. From each project’s total 300 training issues, we assign 30 issues stratified by label as queries and 270 as the retrieval index, mirroring the PS setting. On this validation split, true-question queries retrieve almost balanced bug and question examples: averaged across $k \in [1, 15]$, the mean of $N_{\text{bug}} - N_{\text{question}}$ is -1.36 . We therefore treat a query as a suspected question when $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the top- k and m is a small positive margin. Sweeping $m \in \{1, 2, 3, 4, 5\}$ ⁴ on the validation split and averaging across $k \in [1, 15]$, $m=3$ had a more favorable trade-off against the other values, firing for 89.1% of true-question and 56.7% of true-bug queries. We adopt $m=3$ and refer to the resulting method as **Balanced RAGTAG (BRAGTAG)**.

Every RAGTAG configuration with $k \geq 1$ consistently outperforms the zero-shot and VOTAG baselines **in macro F_1 , precision, and recall**. The best k grows with model size but plateaus within the $k \approx 15$ bound established by VOTAG. RAGTAG is weakest on the question class and frequently misclassifies questions as bugs, motivating a more balanced retrieval method.

C. RQ3: Effectiveness of the Balanced Retrieval Method

Since the PS setting produced the best results for RAGTAG, we use PS as the default for evaluating BRAGTAG on the

⁴We cap the sweep at $m=5$, since it is the 95th percentile of $N_{\text{bug}} - N_{\text{question}}$ on true-question queries.

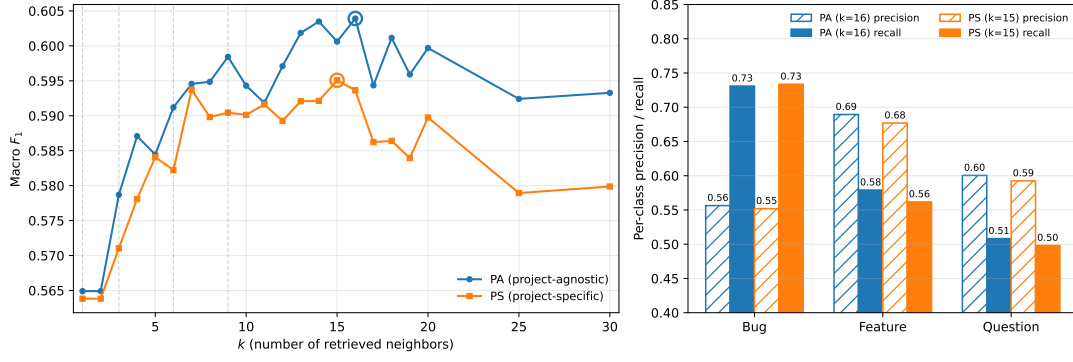


Fig. 3. (left) VOTAG macro F_1 as a function of k in both PS and PA settings. Circles mark peak performance. (right) Per-class precision (hatched) and recall (filled) at each setting's best k ($k=15$ for PS, $k=16$ for PA).

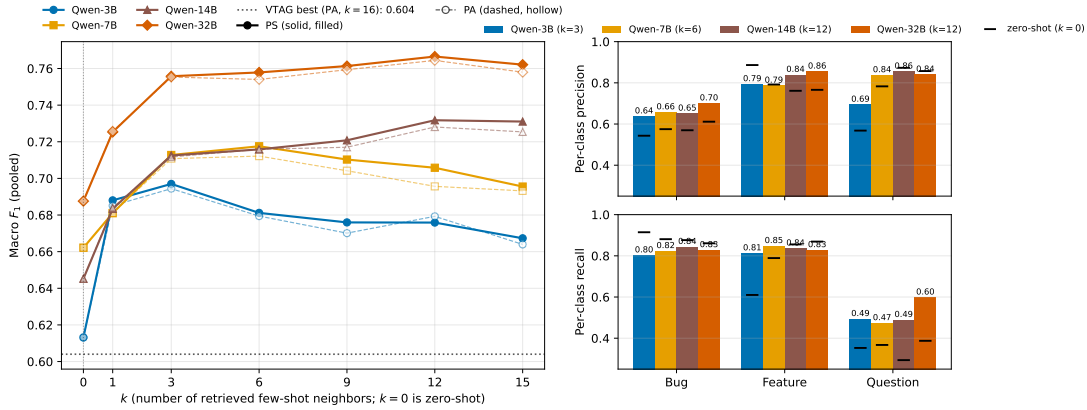


Fig. 4. (left) RAGTAG macro F_1 as a function of k for all four Qwen sizes. $k=0$ is the zero-shot baseline. The dotted horizontal line marks VOTAG's best (0.604, PA $k=16$). (right) Per-class precision (top) and recall (bottom) for each model size at its best RAGTAG-PS configuration; black ticks mark the same model's zero-shot value.

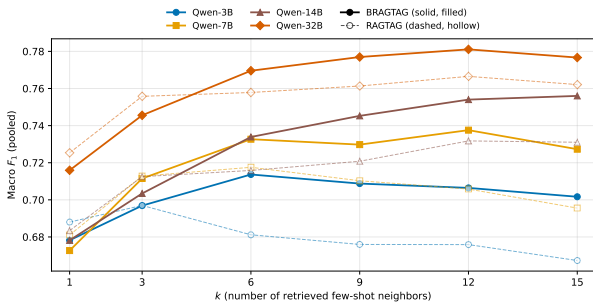


Fig. 5. Macro F_1 as a function of k across all model sizes for both RAGTAG and BRAGTAG (project-specific setting)

same $k \in \{1, 3, 6, 9, 12, 15\}$ grid. Fig. 5 shows macro F_1 vs k for both methods across all four model sizes. BRAGTAG outperforms RAGTAG at every $k \geq 6$. At smaller $k \in \{1, 3\}$, BRAGTAG slightly underperforms by 0.001–0.010 macro F_1 , because the neighbor balancing intervention removes every retrieved example too often, practically reverting the query to zero-shot (40% of queries at $k=1$ and 18% at $k=3$).

BRAGTAG's best k almost always increases compared to

RAGTAG. When the trigger fires, BRAGTAG removes all bug examples from the top- k . Therefore, a higher initial k is needed to leave enough remaining context for the LLM to use. At each method's best k , BRAGTAG outperforms RAGTAG on every model, and the paired bootstrap 95% CI on the (BRAGTAG – RAGTAG) macro F_1 difference excludes zero for all four models⁵. Table I reports the full best-configuration comparison.

The performance gain comes from improvements in the question and bug classes, with minimal impact on feature. Table I reports per-class metrics for each method at its best k . Question-class F_1 improves by 0.030–0.068 across all four model sizes. The improvement comes from recall: question recall rises by 0.065–0.153 (from 0.47–0.60 to 0.56–0.66) at a cost of only 0.012–0.050 in question precision. Bug-class F_1 remains within 0.018 of RAGTAG, with a slight decrease for Qwen-3B and slight gains for the larger models. Thus, removing bug neighbors in selected cases does not reduce overall bug performance. Rather, it shifts bug performance from recall to precision: bug precision rises by 0.045–0.086 while bug recall falls by 0.025–0.138. The largest recall loss is Qwen-

⁵1,000 paired bootstrap resamples per model.

TABLE I
RAGTAG VS. BRAGTAG AT EACH MODEL’S BEST k (PS, POOLED, RAW PREDICTIONS). $P/R/F_1$ ARE PER-CLASS PRECISION, RECALL, AND F_1 ; $P_{\text{MAC}}/R_{\text{MAC}}$ ARE MACRO PRECISION AND RECALL.

Model	Method	k^*	Macro F_1	P_{mac}	R_{mac}	Bug			Feature			Question			Invalid
						P	R	F_1	P	R	F_1	P	R	F_1	
Qwen-3B	RAGTAG	3	0.697	0.709	0.703	0.638	0.803	0.711	0.794	0.813	0.803	0.694	0.494	0.577	0.2%
	BRAGTAG	6	0.714	0.720	0.708	0.723	0.665	0.693	0.794	0.813	0.803	0.644	0.646	0.645	1.8%
Qwen-7B	RAGTAG	6	0.718	0.761	0.715	0.657	0.824	0.731	0.787	0.848	0.816	0.838	0.474	0.605	3.5%
	BRAGTAG	12	0.738	0.766	0.729	0.713	0.774	0.742	0.761	0.855	0.805	0.825	0.558	0.666	3.8%
Qwen-14B	RAGTAG	12	0.732	0.782	0.723	0.651	0.844	0.735	0.839	0.836	0.838	0.855	0.489	0.622	4.5%
	BRAGTAG	15	0.756	0.788	0.742	0.696	0.803	0.745	0.836	0.844	0.840	0.832	0.579	0.683	4.7%
Qwen-32B	RAGTAG	12	0.767	0.799	0.752	0.701	0.828	0.759	0.855	0.828	0.842	0.840	0.598	0.699	4.6%
	BRAGTAG	12	0.781	0.801	0.767	0.748	0.804	0.775	0.847	0.833	0.840	0.808	0.664	0.729	4.0%

3B’s (0.803 to 0.665), the one model where the intervention over-corrects. Feature-class F_1 is essentially unchanged. The question-to-bug misclassification rate also decreases: at each model’s best k , RAGTAG misclassifies 26–36% of true-question issues as bugs, whereas BRAGTAG reduces this rate to 19–27%. Averaged across $k \in [1, 15]$, the rate drops from 31–40% for RAGTAG to 24–36% for BRAGTAG. Question-to-feature misclassification remains unaffected. These results confirm that BRAGTAG’s improvement comes from a targeted reduction in question-to-bug misclassification.

Outputs from both approaches that failed to parse to a valid label are marked as *invalid* and counted as incorrect. Averaged across $k \in [1, 15]$, BRAGTAG has a lower invalid rate than RAGTAG (2.3% vs 3.0% across the four models).

The balanced retrieval method (BRAGTAG) outperforms RAGTAG at every $k \geq 6$. The gain comes from a targeted reduction of question-to-bug misclassifications, raising question F_1 by 0.030–0.068 without sacrificing bug or feature performance.

D. RQ4: Few-Shot Prompting vs. LoRA Fine-Tuning for IRC

Classification Performance.: We fine-tune all four models under both data scopes and compare them with RAGTAG and BRAGTAG, using the best k for each retrieval-based configuration (Fig. 6(a), Table II). When all methods use the same 300 labeled issues per project (PS), both retrieval-based approaches outperform fine-tuning at every model size: RAGTAG leads by 0.021–0.040 macro F_1 , and BRAGTAG by 0.038–0.061. Fine-tuning benefits from pooling the 3,300 labeled issues across all eleven projects (PA). This elevenfold increase in training examples raises its macro F_1 by 0.031–0.084 relative to PS, consistent with prior findings [19]. The larger pool does not provide a corresponding benefit for retrieval (Section V-B). We therefore use PS for RAGTAG and BRAGTAG and PA for fine-tuning in the remaining comparisons, giving each method its better-performing data scope.

We report macro F_1 differences as the retrieval-based method minus fine-tuning, so negative values favor fine-tuning. Aggregated across the four model sizes (13,200 paired predictions),

the difference is -0.029 for RAGTAG (paired bootstrap 95% CI $[-0.037, -0.022]$) and -0.010 for BRAGTAG (CI $[-0.018, -0.003]$).⁶ Fine-tuning is significantly ahead at Qwen-7B and Qwen-14B. At Qwen-3B and Qwen-32B, the confidence intervals include zero, and BRAGTAG has the higher point estimates. Results also vary by project: BRAGTAG outperforms PA fine-tuning in 16 of the 44 comparisons across projects and model sizes, including all four sizes on *opencv* and three of four on *bitcoin* (Fig. 7).

Fine-tuning detects more actual bugs at every model size, while BRAGTAG makes fewer false bug predictions at three of the four sizes. Like the zero-shot models (Section V-B), the PA fine-tuned models over-predict bug: they assign this label to 40–46% of test issues, although only 33% are bugs. Their bug recall is high (0.87–0.93), but precision is lower (0.64–0.73), and 23–36% of question issues are classified as bugs. BRAGTAG predicts bug for 31–39% of issues, closer to the true share, and misclassifies 19–27% of questions as bugs. Its bug recall is lower (0.67–0.80), while its precision is generally higher (0.70–0.75). The one size at which BRAGTAG does not make fewer false bug predictions is Qwen-14B, where fine-tuning’s tendency to over-predict bug is less pronounced: it labels 39.5% of issues as bugs and misclassifies 23% of questions as bugs. Which method is preferable depends on the relative cost of missed bugs and incorrect bug assignments in a given project.

At Qwen-7B and Qwen-14B, BRAGTAG falls behind fine-tuning primarily in recall. Its macro recall is lower by 0.035 and 0.043, respectively, whereas its macro precision is within 0.015 of fine-tuning’s at every model size. Some of these recall errors arise from invalid outputs, which are counted as incorrect predictions and therefore lower recall without affecting precision. At the best k , invalid outputs occur in 0.2–4.6% of cases for RAGTAG and 1.8–4.7% for BRAGTAG. Fine-tuned models, whose training targets specify the required output format, produce invalid outputs in at most 0.8% of cases. These failures also contribute to BRAGTAG’s lower bug recall: at Qwen-7B and larger sizes, 6–8% of true bug issues receive an invalid output.

⁶We use 1,000 paired bootstrap resamples per comparison.

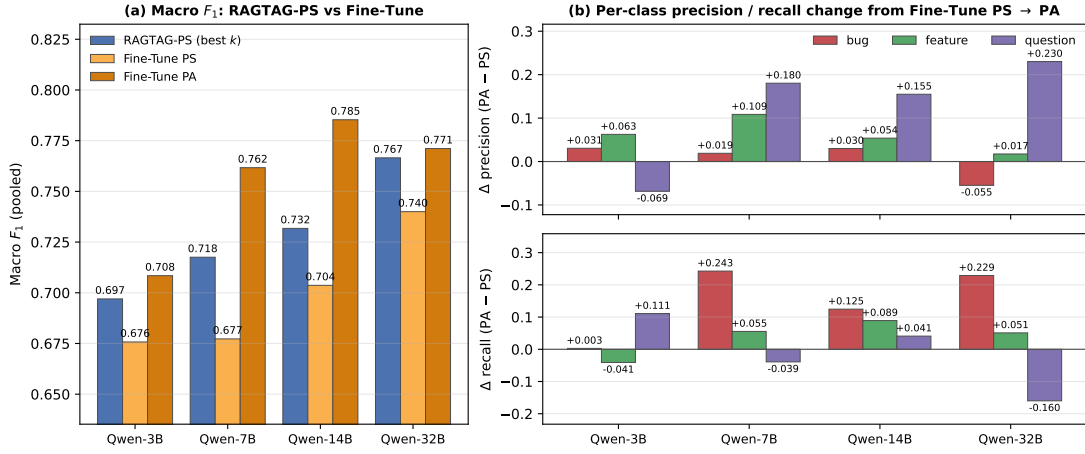


Fig. 6. (a) Macro F_1 of RAGTAG with its best k under the PS data scope, compared with the LoRA fine-tuning baseline under both PS and PA data scopes. (b) Per-class precision change (top) and recall change (bottom) from PS to PA LoRA fine-tuning (PA - PS) for each class and model size.

TABLE II

RAGTAG, BRAGTAG, AND LORA FINE-TUNING AT EACH METHOD’S BEST CONFIGURATION (RAGTAG/BRAGTAG AT THEIR BEST k UNDER THE PS DATA SCOPE, FINE-TUNING UNDER PA; POOLED, RAW PREDICTIONS). P/R ARE PER-CLASS PRECISION/RECALL, P_{MAC}/R_{MAC} THEIR MACRO AVERAGES. THE +VOTAG COLUMN REPORTS MACRO F_1 AFTER USING VOTAG AS A FALLBACK FOR INVALID LLM OUTPUTS.

Model	Method	k^*	P_{mac}	R_{mac}	Bug		Feature		Question		Macro F_1	+VOTAG	Invalid
					P	R	P	R	P	R			
Qwen-3B	RAGTAG	3	0.709	0.703	0.638	0.803	0.794	0.813	0.694	0.494	0.697	0.696	0.2%
	BRAGTAG	6	0.720	0.708	0.723	0.665	0.794	0.813	0.644	0.646	0.714	0.720	1.8%
	Fine-Tune	–	0.731	0.712	0.637	0.873	0.833	0.751	0.724	0.511	0.708	0.711	0.8%
Qwen-7B	RAGTAG	6	0.761	0.715	0.657	0.824	0.787	0.848	0.838	0.474	0.718	0.729	3.5%
	BRAGTAG	12	0.766	0.729	0.713	0.774	0.761	0.855	0.825	0.558	0.738	0.751	3.8%
	Fine-Tune	–	0.780	0.764	0.672	0.878	0.868	0.825	0.800	0.588	0.762	0.762	0.3%
Qwen-14B	RAGTAG	12	0.782	0.723	0.651	0.844	0.839	0.836	0.855	0.489	0.732	0.746	4.5%
	BRAGTAG	15	0.788	0.742	0.696	0.803	0.836	0.844	0.832	0.579	0.756	0.771	4.7%
	Fine-Tune	–	0.792	0.785	0.730	0.865	0.882	0.781	0.764	0.710	0.785	0.785	0.0%
Qwen-32B	RAGTAG	12	0.799	0.752	0.701	0.828	0.855	0.828	0.840	0.598	0.767	0.779	4.6%
	BRAGTAG	12	0.801	0.767	0.748	0.804	0.847	0.833	0.808	0.664	0.781	0.792	4.0%
	Fine-Tune	–	0.805	0.780	0.689	0.929	0.832	0.876	0.895	0.534	0.771	0.771	0.1%

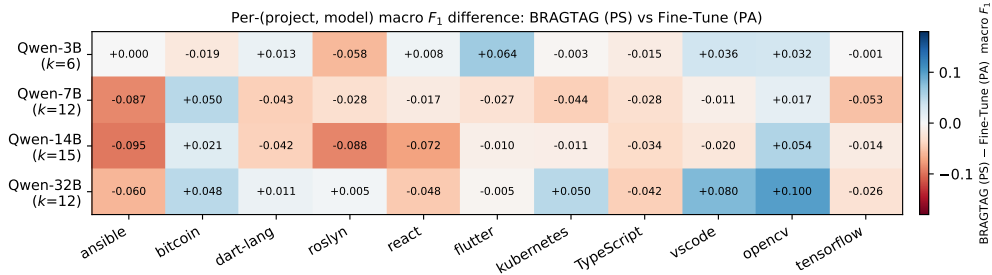


Fig. 7. Per-(project, model) macro F_1 difference between BRAGTAG under the PS data scope and PA LoRA fine-tuning across the 11 projects and four Qwen2.5-Instruct model sizes. Each BRAGTAG result uses the corresponding model’s best k .

To assign a valid label in these cases, we use VOTAG as a fallback for all methods. This requires no additional LLM calls, and the retrieval-based methods can reuse their existing neighbors. We use VOTAG-PS for RAGTAG and BRAGTAG, and VOTAG-PA for fine-tuning (Table II, +VOTAG column). After the fallback is applied to all methods, BRAGTAG’s macro

F_1 deficits at Qwen-7B and Qwen-14B are roughly halved, leaving differences of -0.011 and -0.014 , respectively. At Qwen-32B, BRAGTAG is ahead by $+0.021$. Fine-tuning’s scores change by at most 0.003. The aggregate difference between BRAGTAG and fine-tuning is no longer statistically significant (95% CI $[-0.007, +0.008]$).

Computation and Data Cost.: Our RAG-based methods require 25–47% less peak GPU memory than LoRA fine-tuning across the four model sizes, and remain competitive with it while using $11\times$ less labeled data.

Given the same 300 labeled issues per project, both retrieval-based methods outperform LoRA fine-tuning at every model size (RAGTAG by 0.021–0.040 macro F_1 , BRAGTAG by 0.038–0.061). With $11\times$ more labeled data, fine-tuning leads by 0.029 over RAGTAG and 0.010 over BRAGTAG in aggregate; the lead is confined to Qwen-7B and Qwen-14B and is no longer statistically significant in aggregate once a VOTAG fallback handles invalid outputs. The methods trade different errors: fine-tuning detects more bugs but over-predicts the bug class, whereas BRAGTAG makes fewer false bug predictions.

VI. DISCUSSION

A. Failure Analysis

In this section, we conduct a manual qualitative analysis to better understand the misclassification patterns across the explored IRC methods. We inspect the misclassifications produced by Qwen-32B (the strongest model) under each method’s best configuration. From the full set of misclassifications produced by RAGTAG, BRAGTAG, and LoRA fine-tuning, we randomly sampled 50 misclassified instances per method, yielding 150 instances in total. Each method’s sample includes 30 true-question, 10 true-bug, and 10 true-feature errors. The 30/10/10 split mimics the empirical class distribution of misclassifications pooled across the three methods. Each case was analyzed by two authors independently, with any discrepancies resolved through discussions. The observed failure categories and their frequencies are as follows:

Wrong template format (77.33%).: Issue reports whose content and ground-truth label contradict their chosen template and structure. All of these errors are from questions and feature requests that the user filed under a bug-report template (*Steps to reproduce*, *Expected*, *Actual*, *System Information*). Consequently, the structure of the bug template misleads the LLM classifier towards predicting bug, regardless of the issue’s content. For example, `vscode#193985`⁷ is an actual support question that opens with a literal `Type:Bug` because it uses VS Code’s bug template.

Hybrid intent (13.33%).: Issues that could have multiple defensible labels due to their combined intents. These errors mostly take place when a fix or a feature request is layered into a question or bug report. For example, in `kubernetes#120364`⁸ the author both reports a concrete bug and in the same body explicitly requests a new feature: “Please, give us multiarchitecture image! and fix monoarchitecture images too!”

⁷<https://github.com/microsoft/vscode/issues/193985>

⁸<https://github.com/kubernetes/kubernetes/issues/120364>

Label noise (9.33%).: Issues whose ground-truth label do not match its content. For example, `tensorflow#61710`⁹ is titled “Will there be an MLP model in the future version?” which requests a new API feature for building MLPs. Yet the dataset labels it as a bug.

We also sampled 30 *invalid* predictions each from RAGTAG and BRAGTAG (60 total) and observed three failure modes. In roughly 73% of cases, the model either continued to generate issue-body content or its own reasoning. Another 23% were hallucinations such as repeated digits and punctuation. The remaining 3% are parsable labels that fall outside the three valid classes (e.g., `documentation`).

B. Key Insights and Practical Implications

The practicality of context retrieval for IRC.: Our results indicate that RAG-based approaches offer a more practical solution for IRC than fine-tuning. With $11\times$ less labeled data per project and 25–47% less peak GPU memory, RAGTAG performs competitively against fine-tuning (trails by 0.029 in aggregate macro F_1), and BRAGTAG closes the gap to TOST equivalence at $\delta=0.02$, tightening to statistical equivalence at $\delta=0.01$ under the VOTAG fallback mechanism for invalid LLM outputs (Section V-D). RAG-based methods offer deployment advantages over fine-tuning as well: adding a new labeled issue report becomes an incremental index update rather than a retraining cycle, and the model can be swapped in production with no project-specific retraining. For extremely resource-constrained scenarios, even VOTAG PS is a defensible baseline: it reaches macro $F_1 = 0.595$ ($k=15$), within 0.018 of Qwen-3B’s zero-shot 0.613 (Section V-A, Section V-B), at a fraction of the GPU cost (≈ 240 MB vs 2.9 GB).

Combining classification and validation.: Our failure analysis (Section VI-A) shows that users frequently file support questions and feature requests under the bug-report template, which can bias the classifier towards predicting bug regardless of the content. This emphasizes the importance of refining the new LLM-assisted triage pipelines now in place on platforms such as GitHub [42] and GitLab [43]. In these pipelines, the reporter describes the report to an LLM, which then suggests the correct template and labels. Report validation is also important since mislabeling and duplication is a common problem among contributors [7], [44]. Studies have already explored validation for bug reports [45]–[48]. However, combining classification and validation in a cohesive workflow is underexplored.

VII. THREATS TO VALIDITY

Internal Validity.: BRAGTAG’s margin uses the same empirically derived fixed value across all models and projects with no per-project or per-model tuning, and is not framed as a universally optimal constant. Furthermore, the choice of hyperparameters and prompt format can also pose an internal threat to validity as different hyperparameters and prompts can change fine-tuning performance [6], [19].

⁹<https://github.com/tensorflow/tensorflow/issues/61710>

External Validity.: Issue report text can vary substantially in writing style, vocabulary, and structure across software projects [49], which may threaten the generalizability of our findings. Another threat to external validity is generalizability across settings. We note that many configurations, including LLM architecture, prompt structure, embedding model, and retrieval technique, can each lead to different results. Assessing how far our findings generalize to other settings is itself an important question, but one that falls outside this paper’s scope and warrants its own dedicated studies.

Construct Validity.: The inherent randomness of LLM outputs can pose a potential threat to construct validity. The reported performance metrics can vary between different runs due to LLM randomness. To mitigate this variance, we ran the experiments three times and report the average.

VIII. CONCLUSION AND FUTURE WORK

In this study, we investigated retrieval-augmented few-shot prompting as a cost-efficient alternative to LoRA fine-tuning for issue report classification (IRC). Given a query issue, we retrieved its top- k nearest neighbors from historical issue reports and used them in three proposed approaches: RAGTAG, which presents the neighbors as labeled examples in a few-shot prompt; BRAGTAG, a RAGTAG variant that removes bug-labeled neighbors when the retrieved label distribution suggests that the query is likely a question; and VOTAG, which predicts the label through a weighted vote over the neighbors’ labels, serving as a non-parametric baseline.

We evaluated all three approaches against a LoRA fine-tuning baseline on an 11-project benchmark using Qwen2.5-Instruct models at four sizes. The evaluation considered both project-specific and project-agnostic data scope settings. RAGTAG in the project-specific setting outperforms project-specific LoRA fine-tuning by 0.021–0.040 macro F_1 across all four model sizes and trails project-agnostic LoRA fine-tuning by only 0.029 macro F_1 in aggregate, despite using $11\times$ less labeled data per project. BRAGTAG in the project-specific setting further closes this gap to TOST equivalence at $\delta=0.02$, tightening to statistical equivalence at $\delta=0.01$ under the VOTAG fallback protocol for invalid LLM outputs. [Per-class precision and recall show that BRAGTAG’s gain is a targeted correction of the models’ bug bias, raising question recall by 0.07–0.15 and bringing the share of issues labeled bug from 39–43% to 31–39% \(true share 33%\), a bias that LoRA fine-tuning retains.](#) Averaged across all models, retrieval-augmented few-shot prompting also requires 33% less peak GPU memory than LoRA fine-tuning. Taken together, these results validate our hypothesis: under the studied conditions, low-cost RAG-based approaches can achieve competitive performance with state-of-the-art fine-tuning-based IRC methods, confirming their potential as cost-efficient alternatives.

These findings position retrieval-augmented few-shot prompting as a practical IRC method, particularly when hardware resources are constrained, labeled data is limited, or deployment requires frequent model or data updates. As this work

constitutes a case study on the Qwen2.5-Instruct family, future work can focus on the generalizability of the hypothesis by extending it to other LLM architectures to further strengthen or bound the conclusions drawn here. Additional directions include examining imbalanced label distributions, minority-class scenarios, multi-label classification, and other issue tracking platforms such as Jira and Bugzilla. Further research should also explore stronger retrieval-side interventions, such as LLM-based filtering or re-ranking of retrieved neighbors, as well as the effects of different embedding models and similarity metrics on retrieval quality. Finally, future studies should investigate alternative prompt structures, including advanced strategies such as Chain-of-Thought prompting [50].

DATA AVAILABILITY

Our replication package [51] contains the code, data, and all necessary materials to reproduce the results presented in this paper.

REFERENCES

- [1] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, “Leveraging gpt-like llms to automate issue labeling,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 469–480.
- [2] Z. Liao, D. He, Z. Chen, X. Fan, Y. Zhang, and S. Liu, “Exploring the characteristics of issue-related behaviors in github using visualization techniques,” *IEEE Access*, vol. 6, pp. 24003–24015, 2018.
- [3] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, “Not all bugs are the same: Understanding, characterizing, and classifying bug types,” *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.
- [4] H. Wu, Y. Ma, Z. Xiang, C. Yang, and K. He, “A spatial-temporal graph neural network framework for automated software bug triaging,” *Knowledge-Based Systems*, vol. 241, p. 108308, 2022.
- [5] F. Santos, J. Vargovich, B. Trinkenreich, I. Santos, J. Penney, R. Britto, J. F. Pimentel, I. Wiese, I. Steinmacher, A. Sarma *et al.*, “Tag that issue: applying api-domain labels in issue tracking systems,” *Empirical Software Engineering*, vol. 28, no. 5, p. 116, 2023.
- [6] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, “Applying large language models to issue classification: Revisiting with extended data and new models,” *Science of Computer Programming*, p. 103333, 2025.
- [7] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, and Y.-G. Guéhéneuc, “Is it a bug or an enhancement? a text-based approach to classify change requests,” in *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, 2008, pp. 304–318.
- [8] K. Herzig, S. Just, and A. Zeller, “It’s not a bug, it’s a feature: how misclassification impacts bug prediction,” in *2013 35th international conference on software engineering (ICSE)*. IEEE, 2013, pp. 392–401.
- [9] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, “Ticket tagger: Machine learning driven issue classification,” in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019, pp. 406–409.
- [10] Q. Fan, Y. Yu, G. Yin, T. Wang, and H. Wang, “Where is the road for issue reports classification based on text mining?” in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 2017, pp. 121–130.
- [11] M. Izadi, “Catiss: An intelligent tool for categorizing issues reports using transformers,” in *Proceedings of the 1st international workshop on natural language-based software engineering*, 2022, pp. 44–47.
- [12] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, “Applying large language models to issue classification,” in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 57–60.
- [13] M. Izadi, K. Akbari, and A. Heydarnoori, “Predicting the objective and priority of issue reports in software repositories,” *Empirical Software Engineering*, vol. 27, no. 2, p. 50, 2022.

- [14] A. Trautsch and S. Herbold, "Predicting issue types with sebert," in *Proceedings of the 1st international workshop on natural language-based software engineering*, 2022, pp. 37–39.
- [15] S. Bharadwaj and T. Kadam, "Github issue classification using bert-style models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 40–43.
- [16] G. Colavito, F. Lanubile, and N. Novielli, "Issue report classification using pre-trained language models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 29–32.
- [17] J. Wang, X. Zhang, and L. Chen, "How well do pre-trained contextual language representations recommend labels for github issues?" *Knowledge-Based Systems*, vol. 232, p. 107476, 2021.
- [18] L. Gomes, R. da Silva Torres, and M. L. Côrtes, "Bert-and tf-idf-based feature extraction for long-lived bug prediction in floss: A comparative study," *Information and Software Technology*, vol. 160, p. 107217, 2023.
- [19] J. Heo and S. Lee, "A study on applying large language models to issue classification," in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, 2025, pp. 1–11.
- [20] J. Vargovich, F. Santos, J. Penney, M. A. Gerosa, and I. Steinmacher, "Givemelabeledissues: An open source issue recommendation system," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 402–406.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *Iclr*, vol. 1, no. 2, p. 3, 2022.
- [22] M. Assi, S. Hassan, and Y. Zou, "Llm-cure: Llm-based competitor user review analysis for feature enhancement," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–25, 2026.
- [23] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2438–2449.
- [24] H. Ma, C. Zhang, Y. Bian, L. Liu, Z. Zhang, P. Zhao, S. Zhang, H. Fu, Q. Hu, and B. Wu, "Fairness-guided few-shot prompting for large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2303.13217>
- [25] R. L. L. IV, I. Balažević, E. Wallace, F. Petroni, S. Singh, and S. Riedel, "Cutting down on prompts and parameters: Simple few-shot learning with language models," 2021. [Online]. Available: <https://arxiv.org/abs/2106.13353>
- [26] T. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [27] J. Liu, D. Shen, Y. Zhang, W. B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, 2022, pp. 100–114.
- [28] G. Yu, L. Liu, H. Jiang, S. Shi, and X. Ao, "Retrieval-augmented few-shot text classification," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 6721–6735.
- [29] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [30] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule," *IEEE Transactions on Systems, Man, and Cybernetics*, no. 4, pp. 325–327, 1976.
- [31] G. Colavito, F. Lanubile, and N. Novielli, "Benchmarking large language models for automated labeling: The case of issue report classification," *Information and Software Technology*, p. 107758, 2025.
- [32] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
- [33] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017.
- [34] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [35] J. Von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1487–1507, 2022.
- [36] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Large language models for issue report classification," in *Ital-IA*, 2024, pp. 18–23.
- [37] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.
- [38] H. Huang, R. Widayarsi, T. Zhang, I. C. Irsan, J. Shi, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang *et al.*, "Back to the basics: Rethinking issue-commit linking with llm-assisted retrieval," *arXiv preprint arXiv:2507.09199*, 2025.
- [39] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *IEEE Transactions on Big Data*, 2025.
- [40] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.
- [41] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting," *arXiv preprint arXiv:2310.11324*, 2023.
- [42] GitHub, "GitHub Copilot," <https://github.com/features/copilot>, 2025.
- [43] GitLab, "GitLab Duo," <https://about.gitlab.com/gitlab-duo/>, 2025.
- [44] N. Pandey, D. K. Sanyal, A. Hudait, and A. Sen, "Automated classification of software issue reports using machine learning techniques: an empirical study," *Innovations in Systems and Software Engineering*, vol. 13, no. 4, pp. 279–297, 2017.
- [45] L. Khatib, N. S. Mathews, and M. Nagappan, "Assertflip: Reproducing bugs via inversion of llm-generated passing tests," *arXiv preprint arXiv:2507.17542*, 2025.
- [46] X. Wang, P. Gao, X. Meng, C. Peng, R. Hu, Y. Lin, and C. Gao, "Aegis: An agent-based framework for general bug reproduction from issue descriptions," *arXiv preprint arXiv:2411.18015*, 2024.
- [47] T. Ahmed, J. Ganhotra, R. Pan, A. Shinnar, S. Sinha, and M. Hirzel, "Otter: Generating tests from issues to validate swe patches," *arXiv preprint arXiv:2502.05368*, 2025.
- [48] E. Dinç and E. Tüzün, "Judge the votes: A system to classify bug reports and give suggestions," in *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 2025, pp. 01–10.
- [49] A. Akhavan, A. Hoseinpour, A. Heydarnoori, H. Bagheri, and M. Keshani, "LinkAnchor: An autonomous LLM-based agent for issue-to-commit link recovery," in *To Appear in Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, July 2026, preprint available at <https://arxiv.org/abs/2508.12232>.
- [50] A. Koyuncu, "Exploring fine-grained bug report categorization with large language models and prompt engineering: An empirical study," *ACM Transactions on Software Engineering and Methodology*, 2025.
- [51] "Replication package: Can retrieval-augmented few-shot prompting match lora fine-tuning for issue report classification? an empirical study," <https://figshare.com/s/2c7a6db860c611246774>, Accessed: 05-18-2026.